

[Project Documentation]

RDTH Legal Document Classification System

Automated Legal Text Indicator Mapping using NLP and Machine Learning

March 3, 2026

Junyi Liu

United Nations Economic and Social Commission for Asia and the Pacific

Contents

1	Project Overview	4
1.1	Background	4
1.2	Project Objectives	4
1.3	System Workflow	5
1.4	Key Technologies	5
1.5	Expected Outputs	6
2	System Architecture	6
2.1	Overall Pipeline	6
2.2	Project Repository Structure	7
2.3	Workflow Components	7
3	Data Preparation Pipeline	8
3.1	Raw Legal Document Collection	8
3.2	OCR and PDF Extraction	9
3.3	Text Cleaning	9
3.4	Text Chunking	10
4	Training Dataset Construction	11
4.1	Data Sources	11
4.2	Domain Knowledge Resources	11
4.3	Unified Training Dataset Construction	11
4.4	Training Dataset Description	12
5	Model Training	14
5.1	Model Architecture	14
5.2	Single-Label Classification Model	15
5.3	Multi-Label Classification Model	15
5.4	Optimized Model Training	16
6	Model Evaluation	17
6.1	Evaluation Setup	17
6.2	Evaluation Metrics	18
6.3	Performance Comparison	22
7	Model Testing and Application	28
7.1	Applying Models to New Documents	28
7.2	Single-Text Testing	28
7.3	Full Document Analysis	29

7.4	Output Structure	30
7.5	Confidence Score Interpretation	31
8	Future Improvements	34
8.1	Data Expansion	34
8.2	Model Improvements	34
8.3	Data Cleaning and Chunking Improvements	34
8.4	Domain-Specific Classification Issues (ICT Scope)	35
8.5	Multilingual Extension	35
8.6	Human-in-the-Loop Review	36
8.7	Output and Usability Issues	36
9	References	37
10	Appendix	38

1 Project Overview

1.1 Background

The Regional Digital Trade Integration Index (RDTII) is a framework used to assess countries' readiness for digital trade by evaluating their legal and regulatory environments across multiple policy pillars. These pillars cover areas such as digital regulations, intellectual property protection, electronic transactions, and data governance.

To evaluate these areas, analysts must review a large number of legal and regulatory documents, including national laws, regulations, and policy frameworks. However, manual analysis of these documents is time-consuming and difficult to scale because legal texts are often long and complex.

This project develops an AI-assisted system that uses natural language processing (NLP) to process legal documents and automatically classify legal provisions according to RDTII-related categories. The system helps convert large volumes of legal text into structured information that can support regulatory analysis and policy evaluation.

1.2 Project Objectives

The main objective of this project is to develop an automated system that supports the analysis of legal documents within the RDTII framework.

The system is designed to process large collections of legal and regulatory documents and identify provisions that are relevant to RDTII pillars. By applying natural language processing and machine learning techniques, the project aims to reduce the time required for manual legal document review.

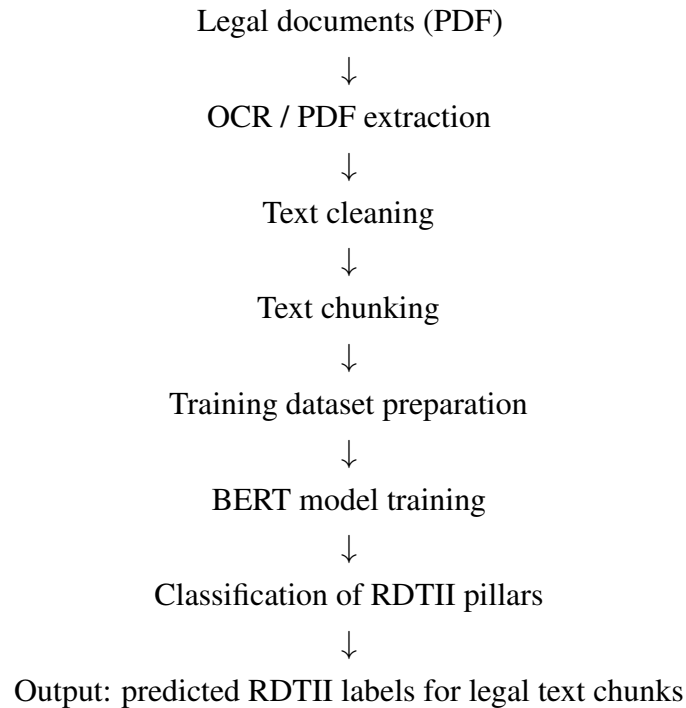
More specifically, the project aims to:

- process raw legal documents and convert them into structured text data
- construct training datasets linking legal text to RDTII-related categories
- train BERT-based classification models to identify relevant regulatory provisions
- apply the trained models to new legal documents for automated analysis

Through this workflow, the system helps transform unstructured legal texts into structured information that can support regulatory analysis and policy research.

1.3 System Workflow

The system follows a step-by-step workflow that converts raw legal documents into structured classification results for RDTII analysis.



This workflow combines document preprocessing, weak supervision, and BERT-based classification to support automated legal text analysis.

1.4 Key Technologies

The system integrates several technologies to support automated legal text classification.

- Natural Language Processing (NLP)
- Transformer-based language models (BERT-based legal models)
- Single-label and multi-label classification
- Text preprocessing, including OCR, cleaning, and chunking
- Python machine learning libraries, including Hugging Face Transformers, PyTorch, and scikit-learn

The roles and implementation of these technologies are explained in more detail in the later sections on the data preparation pipeline and model training.

1.5 Expected Outputs

The system produces structured outputs at different stages of the processing pipeline, from document text extraction to final classification results.

- **Extracted text from legal documents**

Legal documents are initially collected in PDF format. The system converts these files into machine-readable text using PDF extraction or optical character recognition (OCR) when necessary. This step allows scanned or complex legal documents to be processed automatically.

- **Processed text segments (chunks)**

After extraction, the text is cleaned and divided into smaller segments such as paragraphs or sections. These text chunks serve as the basic units for machine learning analysis and classification.

- **Classification results for legal text segments**

The trained models analyze each text chunk and assign predicted labels corresponding to RDTII categories. In the single-label setting, each chunk receives one predicted label, while the multi-label model can assign multiple labels to a single text segment.

- **Structured prediction datasets**

The classification results are exported as structured CSV files. Each record typically includes the document identifier, chunk identifier, predicted label, prediction confidence score, and text segment. These structured outputs allow users to review and analyze classification results across legal documents.

Overall, the system converts raw legal PDFs into structured datasets containing classified legal text segments, supporting further regulatory analysis under the RDTII framework.

2 System Architecture

2.1 Overall Pipeline

The system architecture implements the workflow introduced in Section 1.3. Raw legal documents are converted into machine-readable text, processed into structured text segments, and then classified into RDTII categories using transformer-based models.

The architecture organizes the processing pipeline into several modules, including document preprocessing, dataset preparation, model training, and model application. Each module corresponds to a stage in the machine learning workflow and is implemented through scripts and directories within the project repository.

2.2 Project Repository Structure

The project repository is organized according to the stages of the machine learning pipeline. The structure separates preprocessing scripts, dataset resources, training scripts, and model testing components.

Repository structure:

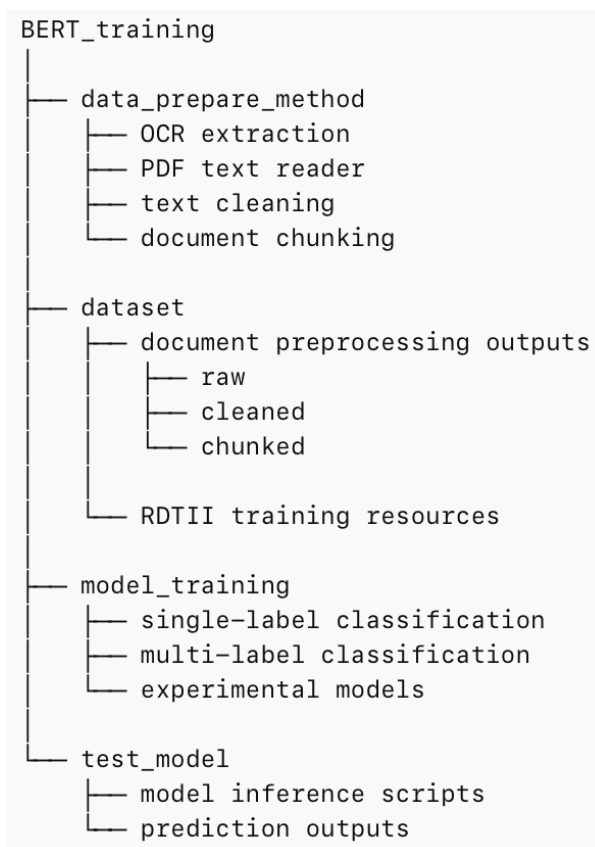


Figure 1: Repository structure of the BERT-based legal text classification system.

2.3 Workflow Components

The repository is organized according to the main stages of the legal text processing pipeline, including document preprocessing, dataset preparation, model training, and model application.

Document preprocessing module

The `data_prepare_method` module converts raw legal documents into machine-readable text. It performs PDF text extraction or OCR when documents contain scanned pages. The extracted text is then cleaned and divided into smaller text segments based on document structure. These segments serve as the basic units used for machine learning classification.

Dataset preparation module

The dataset directory contains both preprocessing outputs and resources used to construct the training datasets. Preprocessing results are organized into stages such as raw documents, cleaned text, and chunked text segments. Additional resources related to the RDTII framework, including the RDTII pillar guide, keyword lists, domain dataset, and other reference materials, are used to support the creation of labeled training data.

Model training module

The training modules implement several classification models based on transformer architectures. These modules use the datasets, tokenize the text segments, and train models under different configurations, including single-label and multi-label classification. The trained models generated during this stage are stored within the training directories.

Model application module

After training, the models can be applied to new legal documents. The `test_model` module loads the trained models and performs classification on processed text segments. The prediction results are exported as structured datasets that contain the predicted RDTII categories for each text segment.

3 Data Preparation Pipeline

3.1 Raw Legal Document Collection

The data preparation pipeline begins with the collection of raw legal documents in PDF format. These documents include national laws, regulations, and policy materials relevant to the RDTII framework. Following the RDTII guide, official legal and government documents are used as the primary sources.

While the system is primarily designed for PDF-based legal documents, some materials may also be obtained from official HTML webpages, which can be collected using web scraping methods and converted into text format before entering the pipeline.

The raw files are stored in the raw directory before preprocessing. Because the documents originate from different jurisdictions and sources, their formats vary. Some PDFs contain machine-readable text, while others are scanned documents that require OCR processing. This stage preserves the original documents and prepares them for text extraction in the next step.

3.2 OCR and PDF Extraction

After raw legal documents are collected, the next step is to convert them into machine-readable text. Because the source PDFs are not uniform, the system applies two extraction methods depending on the document format.

Direct text extraction for digital PDFs

For digital PDFs, the system performs direct text extraction. The PDF reader accesses the internal text layer of the document and retrieves the textual content page by page. This method preserves most of the original characters and layout information. The extracted text is then stored in a structured JSON format containing the document identifier, language field, and page-level text entries.

OCR processing for scanned documents

For scanned or image-based PDFs, the system applies optical character recognition (OCR). Each PDF page is first rendered as an image, and the OCR engine analyzes the image to detect characters and reconstruct words and lines of text. This process converts visual text into machine-readable strings. The recognized text is then saved in the same JSON structure as the direct PDF extraction output so that both document types can enter the same preprocessing pipeline.

Both extraction methods produce structured JSON outputs containing the document identifier, language information, and page-level text. This unified format allows the downstream preprocessing pipeline to treat digital and scanned documents in the same way.

The extracted outputs are stored in the directory:

```
dataset → pre_process_new_data → interim
```

These intermediate JSON files serve as the input for the subsequent text cleaning stage.

3.3 Text Cleaning

After OCR or direct PDF extraction, the intermediate text may contain formatting inconsistencies, irregular line breaks, and OCR artifacts. The text cleaning stage standardizes the extracted text while preserving the structural information needed for later segmentation.

The cleaning process performs several normalization steps. First, Unicode normalization is applied to ensure consistent character encoding. Quotation marks, apostrophes, and dash symbols are standardized to avoid variations in text representation. The script also normalizes

whitespace by collapsing repeated spaces and reducing excessive blank lines while keeping paragraph boundaries intact.

In addition, simple header or footer patterns that commonly appear in legal documents are removed when detected. The script also adjusts the formatting of numbered clauses and legal subsections so that section numbers and clause markers appear on separate lines, making them easier to detect in the following chunking stage.

The cleaned text remains organized at the page level and is stored as structured JSON files. Each file contains the document identifier and the cleaned text for each page. These outputs are saved in the directory:

```
dataset/pre_process_new_data/cleaned
```

This stage prepares the extracted text for structural segmentation while keeping the original document organization.

3.4 Text Chunking

After cleaning, the documents are segmented into smaller analytical units. Legal texts are typically long and structured into sections, clauses, and subsections, so the system divides each document according to these structural patterns rather than splitting the text arbitrarily.

The chunking process scans the cleaned text line by line and detects section headings based on numbered patterns. When a new section heading is identified, a new chunk is created and the previous section is closed. Each chunk contains structured metadata including the document identifier, section number, section title, the starting PDF page, and the list of pages covered by the section.

To improve segmentation quality, simple running headers that appear at the beginning of pages may be removed. The algorithm also preserves paragraph structure within each section so that the full legal context is maintained.

Very short sections that contain only a few words are filtered out, as they are unlikely to provide meaningful information for classification. The remaining sections are written as individual entries in a JSON Lines file, where each line represents one legal text chunk together with its metadata.

The chunked outputs are stored in the directory:

```
dataset/pre_process_new_data/chunked
```

In addition to the main structure-aware chunking pipeline, the project keeps a default balanced chunking version as a backup application method. This fallback is useful when the structure of a new document is not fully captured during the chunking and cleaning process. In such cases, the safer fixed-length chunking method can still produce usable document segments for model inference.

This stage converts cleaned page-level text into structured legal sections that can be used as input units for machine learning classification.

4 Training Dataset Construction

4.1 Data Sources

The training dataset is constructed using both legal text data and domain-specific resources related to the RDTII framework. The materials are stored in the directory:

```
dataset → RDTII_training_ds
```

The main sources include the consolidated legal index data, the official RDTII guide, and supporting reference files used to organize and interpret regulatory provisions.

The **index data** provides structured information extracted from legal sources and includes fields such as the *impact* column, which indicates how specific legal provisions relate to regulatory measures.

4.2 Domain Knowledge Resources

In addition to the index data, domain knowledge resources are incorporated to support the mapping between legal text and the RDTII framework. The **pillar dictionary** and the **RDTII guide** define the conceptual structure of the RDTII indicators and provide the reference taxonomy used for classification.

A **keywords list**, provided by the legal expert team, is also used to link legal provisions to relevant regulatory topics. These keywords help identify semantic relationships between legal text segments and the corresponding RDTII indicators.

Together, these resources provide the domain context necessary to construct a training dataset aligned with the RDTII analytical framework.

4.3 Unified Training Dataset Construction

Using the data sources and domain resources described above, the project constructs a unified training dataset suitable for machine learning.

The dataset integration process is implemented using the pandas library. The index data, keyword resources, and RDTII reference materials are combined and standardized into a single tabular structure. This process produces a consolidated dataset in which each record represents a legal text segment together with its associated RDTII-related information.

The resulting dataset, referred to as the **unified full dataset**, serves as the main input for the model training stage.

This structure corresponds to the organization of the repository resources:

```
RDIII_training_ds
|-- index_data
|-- keywords_list
|-- pillar_dictionary
'-- RDIII_guide
```

4.4 Training Dataset Description

Two versions of the training dataset are used in this project: the semantic extended dataset and the unified full dataset. Both datasets follow the same tabular structure and are constructed from the same group of data sources, but differ slightly in size and composition.

Dataset Structure

Each dataset is organized as a table with three main columns:

- label: the target classification category corresponding to the RDIII framework
- text: the input text used for model training
- source: the origin of the text, indicating which resource it is derived from

Each row represents one training sample, consisting of a text segment paired with a single label.

Semantic Extended Dataset (Model 3)

- Number of observations: 10,427
- Main sources included:
 - act_or_principle
 - index
 - indicator_desc
 - indicator_keyword
 - pillar_keyword
 - pillar_name

This dataset contains structured entries derived primarily from the index data and core domain resources.

Unified Full Dataset (Model 4 and Model 5)

- Number of observations: 10,490
- Main sources included:
 - act_or_principle
 - index
 - indicator_desc
 - indicator_keyword
 - pillar_keyword
 - pillar_name
 - measure
 - sector
 - subject
 - pillar_guide

This dataset includes all elements from the semantic extended dataset, with additional categories that expand the coverage of domain-related information.

Data Sources Description

The **source** column indicates the origin of each text entry. The dataset is constructed from multiple structured files and reference materials used in the RDTII framework.

Sources derived from RDTII Index file

File: RDTII 2.1 Index_new_041125.xlsx

- act_or_principle: Text extracted from the “Act and/or practice” column, describing legal acts, regulations, or policy practices.
- index: Text extracted from the “Impact or comments on Acts or practices” column, providing explanations of regulatory effects.
- indicator_desc: Text derived from the “Category (Policy issue)” column, describing the definition and scope of each indicator.
- pillar_name: High-level RDTII pillar names defined within the index structure.

Sources generated from domain descriptions

- `indicator_keyword`: Keywords generated from the indicator descriptions (`indicator_desc`), summarizing key concepts associated with each indicator.
- `pillar_keyword`: Keywords derived from pillar names and related descriptions, representing broader thematic concepts.

Sources derived from keyword reference file

File: `Tags and Keywords.xlsx`

- `sector`: Extracted from the `Tag_Sector` column, indicating the relevant economic or policy domain.
- `subject`: Extracted from the `Tag_Subject` column, providing additional thematic classification.
- `measure`: Extracted from the `Tag_Measure` column, describing types of regulatory measures or instruments.

Sources derived from RDTII Guide

File: `RDTII Guide 2.1 (Regional Digital Trade Integration Index 2.1: A Guide)`

- `pillar_guide`: Text extracted from the guide, providing explanatory descriptions of pillars and indicators to support contextual understanding.

Summary

Both datasets share the same structure and labeling format. The semantic extended dataset focuses on core structured sources, while the unified full dataset incorporates additional contextual and reference-based inputs, resulting in a slightly larger and more comprehensive training dataset.

5 Model Training

5.1 Model Architecture

The classification models in this project are based on the transformer architecture, using the pretrained Legal-BERT model `nlpaueb/legal-bert-base-uncased`. This model is selected because it is adapted to legal language and is therefore more suitable than general-purpose language models for processing legal text.

Before training, each legal text segment is tokenized and converted into the format required by the transformer model. The tokenized inputs are then used for supervised classification. Depending on the training setting, the model predicts either one label for each text segment or multiple labels for the same segment.

The training pipeline is implemented in Python using Hugging Face Transformers, PyTorch, scikit-learn, pandas, and the datasets library. These tools are used for dataset preparation, tokenization, model training, evaluation, and model saving.

5.2 Single-Label Classification Model

The first successful training configuration is a single-label classification model, implemented in the `stage1_training_3_single_label` module. In this setting, each text segment is assigned one primary RDTII label.

The training dataset used for this model is `unified_semantic_dataset_extended.csv`. Before training, the text and labels are normalized, empty samples are removed, and the labels are converted into numerical identifiers. The dataset is then split into training and testing subsets using stratified sampling.

Tokenization is performed with Legal-BERT using a maximum sequence length of 512 tokens. Since the task is single-label classification, the model is trained with a weighted cross-entropy loss function. Class weights are introduced to reduce the effect of label imbalance in the dataset.

The model is evaluated using accuracy, macro F1, and weighted F1. During training, the best model is selected according to weighted F1 and then saved for later inference.

5.3 Multi-Label Classification Model

The multi-label classification model is implemented in the `stage1_training_4_multilabel` module. Unlike the single-label setting, this model allows one legal text segment to be associated with multiple RDTII labels.

The training dataset for this model is `unified_ds_full.csv`. Labels are transformed into multi-label vectors using binarization so that each sample can contain more than one active label. This representation is appropriate for legal text, where one provision may relate to several regulatory categories simultaneously.

To handle longer legal text more effectively, the model uses tokenization with a sliding-window strategy. When a text segment exceeds the maximum sequence length, it is split into overlapping windows so that more of the original content can be preserved during training.

The model is trained as a multi-label classifier using sigmoid-based outputs. Performance is evaluated using micro F1 and macro F1. Early stopping is also applied in order to reduce overfitting and improve training stability.

5.4 Optimized Model Training

A later training stage introduces an improved single-label model, implemented in the `stage1_training_5_single_label` module. This model still predicts one label per text segment, but it differs from the earlier single-label model in both dataset construction and training strategy.

First, the training dataset is expanded to use `unified_ds_full.csv`, which integrates additional domain resources such as the keyword list and the pillar guide. Compared with the earlier single-label model, this provides richer supervision and a broader representation of RDTII-related categories.

Second, the training procedure is improved through additional preprocessing and model settings. Duplicate text-label pairs are removed before training. The model also adopts a sliding-window tokenization strategy, which makes it better suited for longer legal provisions. Class weights are again used to address imbalance, but the best model is selected based on macro F1 rather than weighted F1. Early stopping is also introduced to improve generalization.

As a result, Model 5 can be understood as an optimized single-label version that builds on the earlier Model 3 while using a richer dataset and a more robust training strategy.

Model	Setting	Dataset	Key Features
Model 3	Single-label	<code>ds_extended.csv</code>	Baseline training with stratified split and Legal-BERT tokenization. Uses weighted cross-entropy and selects the best model based on weighted F1.
Model 4	Multi-label	<code>unified_ds_full.csv</code>	Multi-label binarization with sigmoid outputs and sliding-window tokenization. Early stopping applied. Evaluated using micro and macro F1.
Model 5	Single-label	<code>unified_ds_full.csv</code>	Optimized training using expanded dataset, duplicate removal, sliding-window tokenization, class weighting, and early stopping. Best model selected using macro F1.

Table 1: Comparison of model training strategies

6 Model Evaluation

6.1 Evaluation Setup

This section describes the training configuration and evaluation procedure used to assess model performance. The focus is on the key training parameters and methodological choices that influence how the models learn and generalize.

Training parameters

The training process is controlled by several key parameters:

- **Epochs:** determine how many times the model processes the full training dataset.
- **Batch size:** defines how many samples are processed at one time.
- **Learning rate:** controls how strongly model parameters are updated during training.

In this project, relatively small learning rates and moderate batch sizes are used to ensure stable training for transformer-based models.

Loss function and class imbalance

For the single-label models, training is based on cross-entropy loss. To address the imbalance across RDTII categories, class weights are introduced so that errors on less frequent classes are penalized more heavily. This helps the model avoid overfitting to dominant categories.

For the multi-label model, a binary classification loss is applied independently to each label. This allows the model to evaluate multiple categories for the same text segment.

Handling long text inputs

Legal texts are often longer than the model's input limit. To address this, later models use a sliding-window strategy, where long texts are split into overlapping segments. This allows more information from the original text to be retained during training compared to simple truncation.

Training control and model selection

To improve generalization, early stopping is applied in later models. Training is stopped when validation performance no longer improves, reducing the risk of overfitting.

Model performance is evaluated at the end of each epoch. The best-performing model is selected based on a predefined evaluation criterion (such as F1 score or evaluation loss) and saved for further analysis.

6.2 Evaluation Metrics

This section defines the metrics used to evaluate the performance of the classification model. Since the task involves multiple legal categories with uneven class distributions, several complementary metrics are reported.

Accuracy

The first evaluation metric is accuracy. Accuracy measures the share of all predictions that are correct:

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(\hat{y}_i = y_i)$$

where:

- N is the total number of observations
- y_i is the true label of observation i
- $\hat{y}_i$ is the predicted label
- $\mathbf{1}(\hat{y}_i = y_i)$ equals 1 if the prediction is correct and 0 otherwise

In simple terms, accuracy is the percentage of cases where the model gives the right answer. For example, if the model correctly predicts 80 out of 100 samples, the accuracy is 0.80.

However, accuracy alone can be misleading when the dataset is unbalanced. If one class is very common, a model may achieve high accuracy by predicting that class too frequently.

Precision

Precision answers the question: when the model predicts a category, how often is it correct?

$$\text{Precision} = \frac{TP}{TP + FP}$$

where:

- TP = true positives
- FP = false positives

A high precision means that when the model assigns a legal category, it is usually correct.

For example, if the model predicts that 100 text segments belong to a category, and only 60 of them actually do, the precision is 0.60.

Recall

Recall answers the question: among all texts that truly belong to a category, how many did the model successfully identify?

$$\text{Recall} = \frac{TP}{TP + FN}$$

where:

- FN = false negatives

A high recall means the model misses fewer relevant cases.

For example, if there are actually 100 texts belonging to a category, but the model identifies only 70 of them, the recall is 0.70.

F1-score

The F1-score combines precision and recall into one measure:

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

The F1-score is high only when both precision and recall are high. It is useful because there is often a trade-off between the two.

For example, a model may identify many relevant cases (high recall) but also make many false predictions (low precision), or vice versa.

The F1-score therefore provides a more balanced view of classification quality than accuracy alone.

Macro F1

Macro F1 is the simple average of the F1-scores across all classes:

$$\text{Macro F1} = \frac{1}{K} \sum_{j=1}^K F1_j$$

where:

- K is the total number of classes
- $F1_j$ is the F1-score for class j

This means each class receives equal weight, regardless of how common or rare it is.

Macro F1 is especially important in this project because some legal categories appear much less frequently than others. If the model performs poorly on rare categories, macro F1 will show that clearly.

Weighted F1

Weighted F1 is also an average of class-level F1-scores, but the weights are based on class frequency:

$$\text{Weighted F1} = \sum_{j=1}^K \frac{n_j}{N} F1_j$$

where:

- n_j is the number of observations in class j
- N is the total number of observations

This means larger classes influence the result more strongly than smaller classes.

Weighted F1 reflects overall performance under the true class distribution of the dataset. However, it may hide weak performance on rare labels.

Micro F1 (Multi-label Classification)

For the multi-label model, micro F1 is particularly useful. Instead of averaging class by class, it aggregates all true positives, false positives, and false negatives across all labels:

$$\text{Micro Precision} = \frac{\sum TP}{\sum TP + \sum FP}, \quad \text{Micro Recall} = \frac{\sum TP}{\sum TP + \sum FN}$$

The F1-score is then computed from these aggregated values.

This makes micro F1 more influenced by common labels and overall prediction volume. It is helpful when evaluating the model's total labeling performance across all decisions.

Cross-Entropy Loss

The evaluation loss is the value of the model's loss function on the dataset. For single-label classification, the model uses categorical cross-entropy loss:

$$L(\hat{y}, t) = - \sum_{k=1}^K t_k \log(\hat{y}_k)$$

where:

- t_k is the one-hot encoded true label
- $\hat{y}_k$ is the predicted probability for class k

To address class imbalance, a weighted version is used:

$$L = -w_y \log(p_y)$$

where:

- y is the true class
- p_y is the predicted probability for that class
- w_y is the class weight

This loss function penalizes the model more strongly when it assigns low probability to the correct class. Weighting increases the contribution of underrepresented categories.

Softmax Probability for Single-Label Classification

In the single-label setting, the model produces a score (logit) for each class. These scores are converted into probabilities using the softmax function:

$$p_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$$

where:

- z_j is the model's score for class j
- p_j is the predicted probability

The probabilities sum to 1:

$$\sum_{j=1}^K p_j = 1$$

This means the model must choose among competing classes, and the class with the highest probability becomes the prediction.

Sigmoid Probability for Multi-Label Classification

In the multi-label setting, the model evaluates each label independently using the sigmoid function:

$$p_j = \frac{1}{1 + e^{-z_j}}$$

where:

- z_j is the score for label j
- p_j is the probability that label j applies

Unlike softmax, sigmoid does not force probabilities to sum to 1.

A label is assigned when its probability exceeds a threshold (typically 0.5):

$$\hat{y}_j = \begin{cases} 1 & \text{if } p_j \geq 0.5 \\ 0 & \text{if } p_j < 0.5 \end{cases}$$

This allows the model to associate multiple categories with a single text segment.

Confidence Score

The confidence score reported during prediction is the probability assigned by the model to its prediction.

In the single-label setting, the confidence score is:

$$\text{Confidence} = \max_j p_j$$

In the multi-label setting, the confidence score for each label corresponds to its sigmoid probability.

A higher confidence score indicates that the model is more certain about its prediction. However, confidence does not guarantee correctness.

6.3 Performance Comparison

This section compares the performance of the three training settings and interprets the differences in light of the training scripts, parameter choices, and evaluation strategy.

Overview of training settings

The three models differ not only in classification setting, but also in dataset construction, tokenization strategy, training control, and model selection criterion.

Model 3 is the initial single-label baseline. It uses `unified_semantic_dataset_extended.csv`, stratified train-test splitting, standard Legal-BERT tokenization with a maximum length of 512 tokens, and weighted cross-entropy loss. Training runs for 10 epochs with batch size 4 and learning rate 2×10^{-5} . The best checkpoint is selected using weighted F1.

Model 4 is the multi-label version. It uses `unified_ds_full.csv`, multi-label binarization, sigmoid-based outputs, and sliding-window tokenization with maximum length 512 and stride 256. Training runs for up to 4 epochs with batch size 2 and learning rate 2×10^{-5} . Early stopping is applied, and the best checkpoint is selected using evaluation loss.

Model 5 is an improved single-label design. It also uses `unified_ds_full.csv`, but removes duplicate text-label pairs before training, applies sliding-window tokenization with stride 256, and retains weighted cross-entropy loss. Training is set to 8 epochs with batch size 4

and learning rate 2×10^{-5} , with early stopping enabled. The best checkpoint is selected using macro F1.

These differences matter because they change both the information available to the model and the definition of what counts as the “best” model.

For a more detailed comparison of the training configurations across Models 3, 4, and 5, see Appendix Table 2.

Comparison of reported results

Based on the training outputs provided, the available evaluation results are as follows.

Model	Setting	Epoch reported	Accuracy	Macro F1	Weighted F1	Eval Loss
Model 3	Single-label	10.0	0.5817	0.5776	0.5821	1.6637
Model 5	Single-label	7.0	0.5576	0.5520	0.5566	1.5840
Model 4	Multi-label	—	—	—	—	—

For Model 3, the final screenshot reports:

- evaluation loss = 1.6637
- evaluation accuracy = 0.5817
- evaluation macro F1 = 0.5776
- evaluation weighted F1 = 0.5821

For Model 5, the final screenshot reports:

- evaluation loss = 1.5840
- evaluation accuracy = 0.5576
- evaluation macro F1 = 0.5520
- evaluation weighted F1 = 0.5566

The provided screenshot for Model 5 also shows that training stopped at epoch 7, although the maximum setting was 8 epochs. This is consistent with the use of early stopping.

For Model 4, the training script is available, but no final result screenshot has been provided here. Therefore, its numerical performance cannot be compared directly in this section.

Model 3 versus Model 5

The most direct comparison is between Model 3 and Model 5, since both are single-label models and both report accuracy, macro F1, and weighted F1.

At first sight, Model 5 appears more advanced. Compared with Model 3, it introduces:

- a broader dataset (`unified_ds_full.csv`)
- duplicate removal
- sliding-window tokenization
- early stopping
- model selection based on macro F1 rather than weighted F1

From a methodological perspective, these are meaningful improvements. In particular, sliding-window tokenization is a reasonable response to long legal text, since important information may appear beyond the first 512 tokens. Duplicate removal also reduces redundancy in the training data, and macro F1 is a stricter selection criterion when performance on less frequent categories matters.

However, the reported results show that **Model 5 does not outperform Model 3 on the available evaluation metrics**. Model 3 achieves higher accuracy, higher macro F1, and higher weighted F1 than Model 5. The differences are not extremely large, but they are consistent across all three reported metrics.

This suggests that a more sophisticated training design does not automatically produce better predictive performance. Several explanations are possible.

First, the move from `unified_semantic_dataset_extended.csv` to `unified_ds_full.csv` changes the training distribution. A larger or richer dataset may also be a more difficult dataset. If the additional data introduces more label ambiguity or more heterogeneous examples, performance may decrease even if the model design is more robust.

Second, Model 5 changes the model selection criterion from weighted F1 to macro F1. This means the “best” checkpoint in Model 5 is chosen with more attention to balanced class-level performance rather than overall performance under the empirical class distribution. As a result, the chosen model may be more fair across classes, but not necessarily higher in overall accuracy or weighted F1.

Third, the sliding-window approach increases the number of training instances by splitting long texts into overlapping windows. Although this preserves more content, it also changes the effective training structure. Some windows may contain only partial context, and this may introduce additional noise into the learning process.

Fourth, early stopping in Model 5 may have prevented further optimization. The training stopped at epoch 7, which suggests that later epochs did not improve the chosen validation

metric. This is useful for preventing overfitting, but it may also mean that the final reported model is more conservative.

Overall, Model 5 should therefore be interpreted as an improved training design in methodological terms, but not as a model that clearly exceeds Model 3 in final predictive performance.

Interpretation of Model 3

Model 3 performs best among the models for which complete results are available. This is notable because it is also the simplest design.

Its stronger results may indicate that the baseline setup is already well aligned with the original task structure. The use of stratified splitting, class weighting, and weighted F1-based checkpoint selection may have been particularly suitable for the original dataset. Since the task is highly imbalanced, weighted F1 gives priority to performance under the observed class frequencies, and this may have helped Model 3 achieve a stronger overall score.

Another practical advantage of Model 3 is that its training design is simpler and easier to interpret. It does not introduce sliding windows or early stopping, and its results therefore provide a clear baseline for later experimentation.

Interpretation of Model 5

Although Model 5 scores lower on the available evaluation metrics, it still represents an important experimental development.

Its main contribution is not simply “higher accuracy,” but rather a more robust methodological approach to legal text classification. The use of duplicate removal and sliding-window tokenization responds directly to two common problems in legal NLP:

- repeated or redundant text-label pairs
- long provisions that exceed the transformer input limit

In addition, macro F1-based checkpoint selection reflects a stronger concern for balanced performance across categories. This is especially relevant in legal classification tasks, where rare categories may still be substantively important.

Therefore, Model 5 may still be valuable even if its overall scores are lower in this experiment. It can be understood as a design that emphasizes broader coverage and methodological robustness rather than maximizing the baseline metrics alone.

Interpretation of Model 4

Model 4 differs from Models 3 and 5 because it is a **multi-label classifier**. This means that direct numerical comparison with the single-label models must be made carefully.

The script shows that Model 4 uses:

- `unified_ds_full.csv`
- multi-label binarization
- sigmoid output
- threshold-based prediction
- sliding-window tokenization
- early stopping
- evaluation based on micro F1 and macro F1

These choices are appropriate for settings in which a legal text segment may relate to more than one category. In conceptual terms, Model 4 is important because legal provisions often overlap across several regulatory domains.

At the same time, one technical point should be stated carefully. In the provided script, labels are transformed with:

```
df["label_list"] = df["label"].apply(lambda x: [x])
```

This means each row is converted into a one-element label list before binarization. So while the model architecture and output layer are set up for multi-label classification, the training data shown in the script does not clearly demonstrate multiple simultaneous ground-truth labels per row. For this reason, Model 4 is best described as a multi-label training formulation or a multi-label-capable setup, rather than conclusive evidence that the dataset fully uses genuine multi-label annotation at sample level.

Since no final evaluation output is shown here, Model 4 should be interpreted mainly at the methodological level in this section.

Parameter review and implications

The training parameters are generally consistent across models, which helps isolate the effect of design changes.

The learning rate is fixed at 2×10^{-5} in all three models. This is a standard conservative choice for fine-tuning transformer models and supports stable optimization.

The batch size is 4 in Models 3 and 5, but reduced to 2 in Model 4. This smaller batch size is reasonable because sliding-window tokenization and multi-label outputs increase memory demand.

The maximum sequence length is 512 in all models. This is the standard upper bound for BERT-like models and is appropriate for legal text.

The stride parameter of 256 appears only in Models 4 and 5. This means overlapping windows retain more long-text context, but they also increase training complexity and the number of effective samples.

The epoch setting differs across models:

- Model 3: 10 epochs
- Model 4: 4 epochs
- Model 5: 8 epochs maximum

These differences likely reflect both experimentation and the addition of early stopping in the later models.

The loss function also differs by task structure:

- weighted cross-entropy in Models 3 and 5
- binary cross-entropy through multi-label formulation in Model 4

This is theoretically appropriate given the distinction between single-label and multi-label classification.

Finally, the best-model selection criterion changes across models:

- Model 3: weighted F1
- Model 4: evaluation loss
- Model 5: macro F1

This is important for interpretation. Since each model is optimized toward a different target, the reported outputs do not represent exactly the same training objective. This partly explains why direct comparison should be made with caution.

Overall assessment

Taken together, the results suggest three main conclusions.

First, **Model 3 provides the strongest baseline performance** among the models with complete reported outputs. Its results are the highest on accuracy, macro F1, and weighted F1.

Second, **Model 5 introduces important methodological improvements**, especially for longer legal texts and duplicate handling, but these improvements do not translate into higher reported evaluation scores in the current experiment.

Third, **Model 4 expands the framework toward multi-label classification**, which is conceptually valuable for legal text analysis, but its current script and the absence of final evaluation results mean that its practical performance cannot yet be assessed as clearly as the single-label models.

Therefore, the model development process should not be understood as a simple linear progression from weaker to stronger performance. Instead, it reflects a series of experiments that balance predictive accuracy, class fairness, long-text handling, and task formulation. In that sense, the comparison across Models 3, 4, and 5 provides useful methodological insight even when later versions do not uniformly outperform the initial baseline.

7 Model Testing and Application

7.1 Applying Models to New Documents

After training, the saved models are applied to new legal and policy documents through dedicated inference scripts. These scripts load the trained model, tokenizer, and label mapping, and then generate predictions for unseen text.

For Model 3 and Model 5, the application logic is the same because both are single-label classification models. The input text is tokenized and passed through the trained classifier. The model outputs logits across all possible labels, and these logits are converted into probabilities using the softmax function. The label with the highest probability is selected as the final prediction, and the corresponding probability is reported as the confidence score.

For Model 4, the application logic differs because it is a multi-label classification model. Instead of selecting only one class, the model applies the sigmoid function to the output logits, producing an independent score for each label. The labels are then ranked by probability, and the top predicted labels are returned. In the current implementation, the top three labels are retained for each text segment.

This means that Model 3 and Model 5 produce one final RDTII label per input, while Model 4 can retain several likely labels for the same input.

7.2 Single-Text Testing

The most direct form of testing is to apply the trained model to a single paragraph or short regulatory passage.

For Model 3 and Model 5, the single-text test follows the same procedure. The input paragraph is tokenized and passed to the model, which returns one predicted label and one confidence score. This provides a simple way to inspect whether the model assigns a reasonable RDTII category to an isolated piece of text.

For Model 4, the same input is processed differently. Since the model is multi-label, it returns several likely labels ranked by score instead of a single prediction. This is useful when one passage may relate to more than one policy issue.

The key difference at this level is therefore that Model 3 and Model 5 assign one label to each input, whereas Model 4 can assign multiple ranked labels.

7.3 Full Document Analysis

For full document analysis, the models are applied not to the entire legal document as one block, but to document chunks created during preprocessing. In this project, two chunking approaches are used for application: a structure-aware method and a default balanced method.

Smart Chunked Document Analysis

The main document-level application uses structured chunks generated through the chunking process described in Section 3.4. In this approach, the text is segmented according to legal structure, such as sections, articles, or headings, rather than by arbitrary length alone. The chunking script checks the internal structure of the cleaned legal text and creates chunks that preserve legal boundaries as much as possible.

Each structured chunk contains the text together with metadata such as the document identifier, PDF page references, section number, and section title.

For Model 3 and Model 5, each chunk is passed through a single-label prediction function. The output probabilities are produced through softmax, and the most likely label is selected. The final output includes the chunk metadata, the predicted label, and the corresponding confidence score.

For Model 4, the same structured chunks are processed using a multi-label prediction function. The model produces sigmoid probabilities for all labels, and the top-ranked labels are retained for each chunk. In the current implementation, the output stores the top three predictions and their scores for each chunk.

This structure-aware approach is the main application method because it aligns the classification unit with the legal organization of the document.

Default Chunked Document Analysis

A second chunking method is also retained as a fallback. In this approach, cleaned text files are divided into balanced character-based chunks using fixed-size segments with overlap. In the current implementation, each chunk contains approximately 1000 characters with 150 characters of overlap between adjacent chunks.

This method does not rely on detecting section headings or article boundaries. Instead, it provides a more robust segmentation strategy when the structure of a new legal document cannot be reliably captured by the structure-aware chunking script. This may occur when a document uses an unfamiliar layout, inconsistent numbering style, or formatting that is not fully recognized during preprocessing.

Each chunk is stored with basic metadata, including the document identifier, chunk identifier, character start position, character end position, and the text itself.

These chunks can then be passed to the same prediction pipeline used for full document inference.

This backup design improves the practical robustness of the system and is directly linked to the preprocessing and chunking issues discussed in Section 3.3 and Section 3.4.

7.4 Output Structure

The prediction results generated by the models are saved as CSV files, which provide a structured and human-readable format for further analysis. These files are produced directly from the Python inference scripts during model execution.

Single-Label Output (Model 3 and Model 5)

For Model 3 and Model 5, each row in the output file corresponds to one text unit (either a paragraph or a chunk). The structure typically includes:

- doc_id: identifier of the legal document
- pdf_pages: page references from the original PDF
- section: section number
- title: section title
- text: the original text content
- predicted_label: the assigned RDTII indicator
- confidence: the probability score of the prediction

Each text unit is assigned one label only, reflecting the single-label classification logic. The confidence score is derived from the softmax probability of the selected class.

Multi-Label Output (Model 4)

For Model 4, the output structure is extended to support multi-label predictions. Each row still corresponds to one text unit, but instead of a single label, the output includes multiple ranked predictions:

- prediction_1, score_1
- prediction_2, score_2
- prediction_3, score_3

These columns represent the top predicted RDTII indicators and their associated probabilities. The probabilities are obtained using the sigmoid function and reflect the independent likelihood of each label.

This structure allows one text segment to retain multiple relevant regulatory interpretations.

File Generation and Practical Usage

The CSV files are generated in the Python environment (e.g., PyCharm) and serve as the main output of the model application stage.

In practice, the files can be opened successfully using Mac Numbers, where the table structure is preserved clearly. This allows users to:

- sort results by confidence score
- filter by predicted labels
- identify the highest-confidence predictions
- manually inspect the correspondence between text and predicted indicators

This workflow provides an efficient way to support human validation and interpretation of model outputs.

Practical Issue: Excel Compatibility

When opening the CSV files in Microsoft Excel with default settings, formatting issues may occur, such as misaligned columns or incorrectly displayed text.

This problem is likely caused by encoding differences (UTF-8) or delimiter interpretation in Excel. As a result, Excel may not correctly parse the file without manual import configuration. Therefore, for reliable usage:

- Mac Numbers and pycharm are currently used as the primary tools for viewing results
- Excel requires manual adjustment (e.g., encoding selection or import settings) and is not used as the default viewing tool in this project

7.5 Confidence Score Interpretation

The confidence score reported in the output corresponds to the probability assigned by the model to each prediction. The computation of these probabilities (softmax for single-label and sigmoid for multi-label classification) has been introduced in Section 6.2 (Evaluation Metrics) and is not repeated here.

Generation in the Inference Process

During inference, the model produces raw outputs (logits), which are converted into probabilities and then used to generate predictions.

For Model 3 and Model 5 (single-label):

- probabilities are normalized across all labels
- the label with the highest probability is selected
- the confidence score is the probability of this selected label

For Model 4 (multi-label):

- each label is assigned an independent probability
- probabilities are not normalized across labels
- labels are ranked by probability
- the top-k labels (e.g., top 3) are selected, each with its own score

In full document analysis using sliding-window inference, probabilities are first computed for each window and then aggregated (averaged) across windows. The final confidence score therefore reflects the overall relevance of the entire text segment.

Typical Range of Scores

In practice, the scale of confidence scores differs between models.

- For single-label models, scores are generally higher, often in the range of 0.6–0.9, and sometimes above.
- For multi-label models, scores are typically lower, often below 0.4, with many relevant labels appearing around 0.1–0.3.

This difference is due to the underlying probability structures and the complexity of legal text, where multiple indicators may be relevant simultaneously.

As a result, a fixed threshold (such as 0.5) is not appropriate in this project.

Interpretation and Usage

Confidence scores should be interpreted as relative indicators of relevance, rather than absolute measures of correctness.

- Higher scores indicate stronger alignment between the text and the label

- Moderate scores may still represent meaningful relationships
- Very low scores (close to 0.00) are typically less relevant

Instead of applying a fixed threshold, a ranking-based approach is used. Predictions are ordered by confidence, and the highest-ranked labels are considered first.

Practical Workflow for Document Analysis

In practice, the confidence score is mainly used to support efficient document review. A typical workflow is:

1. Rank all predicted results within a document by confidence score
2. Focus on the top-ranked sections or chunks
3. Compare these sections with the predicted labels
4. Identify the main regulatory mappings

If the highest-ranked results are already meaningful and consistent, it is often not necessary to examine all remaining low-confidence segments. This allows efficient analysis of large legal documents.

Combining Single-Label and Multi-Label Results

In this project, single-label and multi-label outputs can be used together:

- single-label results provide a clear primary classification
- multi-label results provide additional candidate labels

Empirically, labels with high confidence in the single-label model often also appear among the top-ranked results in the multi-label model, although with lower scores.

Limitations

Confidence scores may vary across different legal documents due to differences in structure, language, and content. Therefore, scores are not directly comparable across documents.

The score should not be interpreted as an absolute measure of correctness, but rather as a tool for relative ranking within each document.

8 Future Improvements

8.1 Data Expansion

The dataset can also be improved at the data level.

A first direction is to increase the overall amount of labeled data. More examples from different legal documents and jurisdictions would help the model capture a wider variety of regulatory language.

Second, the training dataset design can be strengthened. The current training data can be expanded with more examples for each indicator, especially for underrepresented labels. It would also be useful to include more negative examples, so that the model can better distinguish between text that is relevant to an indicator and text that is semantically similar but not actually applicable.

Third, the multi-label structure of the dataset can be improved substantially. At present, the multi-label model is trained on data where each row still corresponds to only one label. A stronger multi-label training design would require examples in which the same paragraph or legal section is explicitly linked to multiple labels. This would allow the model to learn true multi-label relationships rather than simply ranking several possible labels from a largely single-label dataset.

8.2 Model Improvements

Although the current models provide a workable classification pipeline, several aspects of model training can still be improved.

First, further experiments can be conducted on training parameters, including the number of epochs, batch size, and learning rate. These settings may influence model stability and performance across different datasets.

Second, the project can explore more robust pretrained models and alternative NLP architectures. While Legal-BERT provides a strong baseline, it may be useful to test other transformer-based models in order to compare robustness, generalization ability, and performance on long legal text. For example:

- Longformer for longer text sequences
- RoBERTa or DeBERTa (including Legal-RoBERTa) for stronger robustness

8.3 Data Cleaning and Chunking Improvements

The preprocessing pipeline can also be further improved.

Legal documents from different jurisdictions often have different structural patterns, including variations in numbering systems, section markers, and formatting styles. As a result, the current rule-based chunking approach may not always correctly capture document structure.

Future work should focus on:

- improving structure detection rules
- adapting chunking logic to different document formats
- making the pipeline more flexible across diverse legal texts

8.4 Domain-Specific Classification Issues (ICT Scope)

An important issue observed in this project is related to domain specificity.

The model often correctly identifies legal measures, but may misclassify the sector or product domain. For example, regulations concerning non-ICT goods (such as agricultural products) may still receive high confidence scores because the legal structure resembles ICT-related regulations.

This reflects the nature of transformer models, which rely heavily on semantic similarity and contextual patterns, rather than strict domain boundaries.

A further direction is to improve the balance and quality of the dataset. In particular, the current system may still classify text as relevant when the regulatory measure matches, even though the actual product domain is outside the intended RDTII scope.

This suggests that the dataset should better distinguish between:

- text that is legally similar in structure, and
- text that is substantively relevant to ICT-related regulation

Possible improvements include adding more domain-specific negative examples, strengthening the representation of ICT-related terminology, or increasing the importance of key sector-specific terms in the training data by modifying the training weight for specific words.

8.5 Multilingual Extension

Another possible direction is multilingual extension.

At present, the project is mainly designed for English legal text after extraction and cleaning. However, legal materials in real-world applications may appear in different languages or in bilingual formats.

Future work could therefore extend the system to multilingual legal document processing. This may involve:

- multilingual text preprocessing
- multilingual legal embeddings or transformer models

- XLM-RoBERTa or mBERT for multilingual legal documents
- parallel indicator mappings across languages

8.6 Human-in-the-Loop Review

Human review remains important for the practical use of the system.

The current models are useful for quickly scanning documents and highlighting likely relevant sections. In practice, this already improves efficiency, since users can rank sections by model score and focus on the top predictions first. However, the model output should still be reviewed by a human.

A human-in-the-loop workflow allows the model to act as a first-pass screening tool, while final validation is conducted by a human reviewer.

This is especially important when:

- document structures are complex or inconsistent
- chunking does not fully capture legal sections
- predictions are close but not fully correct
- domain mismatches occur

Even when the model is not perfectly accurate, it can still significantly reduce manual workload by helping users quickly identify the most relevant sections.

8.7 Output and Usability Issues

Some practical issues arise in the output stage.

Currently, model outputs are exported as CSV files, which work well in tools such as Mac Numbers. However, compatibility problems may occur in other software (such as Microsoft Excel), where encoding issues (such as UTF-8) may lead to incorrect display or formatting.

Future improvements may include:

- standardizing export formats and encoding
- improving compatibility across spreadsheet tools
- developing a more user-friendly interface for result exploration

9 References

Maastricht Law & Tech Lab. *Awesome Legal NLP Tools*. <https://github.com/maastrichtlawtech/awesome-legal-nlp>

Natural Language Processing for the Legal Domain: A Survey of Tasks, Datasets, Models, and Challenges. <https://arxiv.org/html/2410.21306v3>

LexPredict. *LexNLP: Natural Language Processing for Legal Text*. <https://github.com/LexPredict/lexpredict-lexnlp>

ICLR & UCL. *Blackstone: Legal NLP Pipeline*. <https://github.com/ICLRandD/Blackstone>

World Bank. *WB NLP Applications*. <https://github.com/worldbank/wb-nlp-apps>

World Bank. *LLM4Data*. <https://github.com/worldbank/llm4data>

Hoom, Kh. *Cat Recognition with Deep Learning*. <https://github.com/HoomKh/Cat-Recognition-with-DeepLearning>

Roh, D. *Machine Learning Projects*. <https://github.com/DorsaRoh/Machine-Learning>

“Classifying legal interpretations using large language models”. <https://doi.org/10.1007/s10506-025-09447-9>

***Processing Long Legal Documents with Pre-trained Transformers*.** <https://arxiv.org/abs/2211.00974>

***Scaling Legal AI: Benchmarking Mamba and Transformers*.** <https://arxiv.org/abs/2509.00141>

Bayona, P. *Beyond Six Digits: Automated Tariff Line HS Transposition Using NLP*. https://www.wto.org/english/res_e/reser_e/ersd202504_e.htm
https://www.wto.org/english/res_e/reser_e/ersd202504_e.pdf

Alajmi, K., Deodoro, J., Khan, A., & Moriya, T. *Predicting the Law*. <https://www.imf.org/en/Publications/WP/Issues/2023/11/18/Predicting-the-Law-Artificial-Intelligence-Findings-from-the-IMFs-Central-Bank-Legislation-541619>

Chalkidis, I., Jana, A., Hartung, D., Bommarito, M., Androutsopoulos, I., Katz, D., & Aletras, N. *LexGLUE Benchmark*. <https://doi.org/10.48550/arXiv.2110.00976>

Dyson, A. *Text-mining for Lawyers*. <https://repub.eur.nl/pub/137009/ELR-D-20-00035.pdf>

10 Appendix

Component	Model 3 (Baseline)	Model 4 (Multi-label)	Model 5 (Optimized)
Setting	Single-label	Multi-label	Single-label
Dataset	ds_extended.csv	unified_ds_full.csv	unified_ds_full.csv
Label Format	Single class per sample	Multi-label binarized (list $\rightarrow$ vector)	Single class per sample
Tokenization	Standard BERT tokenization	Sliding window (max_len=512, stride=256)	Sliding window (max_len=512, stride=256)
Handling Long Text	Truncation at 512 tokens	Overlapping windows	Overlapping windows
Duplicate Handling	Not applied	Not explicitly applied	Duplicate removal applied
Train/Test Split	Stratified split	Random split	Random split
Batch Size	4	2	4
Learning Rate	2×10^{-5}	2×10^{-5}	2×10^{-5}
Epochs (max)	10	4	8
Early Stopping	No	Yes	Yes
Loss Function	Weighted cross-entropy	Binary cross-entropy (multi-label)	Weighted cross-entropy
Class Imbalance Handling	Class weights	Implicit via BCE	Class weights
Output Activation	Softmax	Sigmoid	Softmax
Prediction Type	One class only	Multiple labels possible	One class only
Model Selection Criterion	Weighted F1	Evaluation loss	Macro F1
Evaluation Metrics	Accuracy, Macro F1, Weighted F1	Micro F1, Macro F1	Accuracy, Macro F1, Weighted F1
Key Design Focus	Stable baseline	Multi-label flexibility	Data quality + long-text handling

Table 2: Detailed Comparison of Model Training Configurations